AB Power Meter Monitor 1.7.0 - Windows test kit

William W. Wallace, NRAO/GBO

2026-10-27

AB Power Meter Monitor 1.7.0 — Windows test kit

Everything needed to test the monitor on a Windows 10/11 PC (first run on **txrx03**, Windows 11 26H2, i9, 2026-10-26). No real meters are needed: a built-in emulator serves the 11 meter pages for two meters on `127.0.0.2` and `127.0.0.3`. Nothing outside the kit folder is changed; delete the folder to remove everything.

New in 1.7.0 / kit 1.2.0

The GUI plots are **interactive** (pyqtgraph, `ab_live_plots.py`): one tab per meter, one plot per unit, a crosshair whose values are listed beside every plot, zoom and pan while the data update, and the meter's identity (address it reports, serial number) at the top of its tab. Checklist: step 4 below.

What	Do
Kit environment	`environment-windows-test.txt` 1.2.0 adds `pyqtgraph=0.14.0` (still OpenBLAS). New folder: run step 1. Existing `env\`: `mamba install -y -p .\env -c conda-forge --override-channels pyqtgraph=0.14.0`
Your `py3p12`	`mamba install -n py3p12 -c conda-forge pyqtgraph` — without it the GUI still works, with the old pictures, and the console says so
Step 2	eleven suites: + `live_plots` (every value drawn and every crosshair read-out equals the stored value of that meter; zoom, scroll, no leaks, fallback)
Files	keep `ab_live_plots.py` next to `ab_power_meter_monitor.py`

New in 1.6.4 / kit 1.1.1

Prometheus exporter 0.2.0 and the Grafana dashboard (see `monitoring\README_monitoring.md`); step 2 runs two more suites, `grafana` (dashboard builder) and `ip_separation` (every meter's data stays with that meter, 1.6.5). Nothing else changed for Windows.

New in 1.6.3 / kit 1.1.0 (from the txrx03 results)

Found on txrx03	Fix
GUI closed with exit code `-1066598273` (0xC06D007F) on the first plot	NumPy's matrix library, Intel oneMKL 2026.1.0, crashed inside `np.dot` on that CPU. The kit now installs **OpenBLAS** (`libblas=*=*openblas`), confirmed on txrx03: 80 polls, every output written, no warning.
OpenBLAS showed ~3 GB of commit charge (2,971 MB)	The monitor sets `OPENBLAS_NUM_THREADS=1` before NumPy loads (measured 18 MB). A value you set yourself is kept.

Found on txrx03	Fix
First poll in step 5 refused by the emulator	Steps 4 and 5 now wait until the emulator answers (`wait_for_port.py`, max 30 s).
An offline meter cost ~67 s and 44 warnings per poll	Its remaining pages are skipped once it refuses the connection (`SKIP_PAGES_WHEN_UNREACHABLE`).
“final writes: up to date” when no meter ever answered	Now “no data received”.

Already have the 1.6.2 kit environment? Re-running step 1 into a new folder is the clean way. To switch an existing `env\` in place instead (PowerShell or cmd, kit folder): `mamba install -y -p .\env -c conda-forge --override-channels "libblas=*=*openblas"`

PowerShell or cmd

Double-clicking the `.bat` files is simplest. From a terminal:

	PowerShell	cmd
Run a step	`.\4_gui_with_emulator.bat`	`4_gui_with_emulator.bat`
Kit Python	`.\env\python.exe …`	`env\python.exe …`
Change folder	`cd C:\path`	`cd /d C:\path`
Start in a new window	`Start-Process .\env\python.exe -ArgumentList …`	`start "" env\python.exe …`

No conda environment needs to be active: the kit always runs its own `env\python.exe`. Run `mamba deactivate` until the prompt shows no `(base)` / `(py3p12)` if in doubt.

Your own environments (e.g. `py3p12`)

The monitor also runs in your own conda environment. If its NumPy uses oneMKL 2026.1.0 the GUI can crash the same way; switch it with `mamba install -n py3p12 -c conda-forge "libblas=*=*openblas"`. (`py3p12` currently has MKL 2026.2.0, which works on txrx03 but has been marked *broken* on conda-forge, so a future update may replace it.)

What it checks

Area	How
RAM flush gives memory back to **Windows** (1.6.1)	`windows_memory_check.py` reads the working set and commit charge with `GetProcessMemoryInfo`; step 3 lets you watch it in Task Manager
Output files held open by **Excel** (1.6.2)	`test_windows_file_locks.py` locks the CSV, Markdown, XLSX and FITS files the way Excel does (`CreateFileW`, read-share only)
Ctrl+Break stops the headless monitor and the exporter cleanly (1.6.2 / 0.1.1)	headless soak and exporter test send `CTRL_BREAK_EVENT`
No row lost or written twice, in every output, across flushes	offline suite + headless soak (CSV = Markdown = .vszh5 = XLSX = FITS row counts)
No leak over time	GUI soak (widgets, RSS, handles, GDI/USER objects) and headless soak (working set, handles, threads)
Python 3.12 on Windows	every suite, with this project’s DeprecationWarnings as errors

Before you start

1. **Unblock the 7z** before extracting: right-click it → *Properties* → tick *Unblock* → *OK*. (Otherwise Windows asks before every `.bat`.)
2. **Use a short path without spaces**, e.g. `C:\ABTest` or the ready copy in `C:\Users\wwallace\Claude\AB_Monitor_1.6.3_`. Conda environments break on paths with spaces; the setup refuses them.
3. Have **Miniforge** (mamba) installed — the setup also finds conda, micromamba, Miniconda or Anaconda. Without any of them it falls back to python.org Python 3.12 (`py -3.12`) with pip.
4. About **3 GB** free disk and an internet connection for step 1.

Steps

Step	Double-click	Time	What happens
1	`1_setup_environment.bat`	5–10 min	creates `env\` (Python 3.12.15 and the exact package versions tested); asks whether to add `env_veusz\` (Veusz 4.2.1, for the real-Veusz checks and "Open in Veusz") — answer **Y** unless Veusz is installed
2	`2_run_all_tests.bat`	~20 min	all eleven suites; writes `results\<computer>_<time>\` and **`results\AB_TestResults_<computer>_<time>.zip`**. `2_run_all_tests.bat --quick` takes ~8 min
3	`3_task_manager_check.bat`	~3 min	six flush runs, each pausing 20 s so you can watch Task Manager
4	`4_gui_with_emulator.bat`	~10 min	the real GUI against the emulator — follow the checklist below
5	`5_headless_with_emulator.bat`	~3 min	headless run with a visible log; stop it with **Ctrl+C** or **Ctrl+Break**

Longer soak (optional, e.g. over lunch): `2_run_all_tests.bat --only headless_soak --soak-minutes 60`

Step 3 — what to look for

Task Manager → *Details* → right-click a column header → *Select columns* → tick **Working set (memory)** and **Commit size**. Each run prints its PID. After the flush of a `full` run the working set W should fall back close to where it started:

$$W_{\text{after flush}} \leq W_{\text{idle}} + 0.3\,(W_{\text{filled}} - W_{\text{idle}})$$

`gc_only` is how 1.6.0 behaved on Windows (rows dropped, memory kept); `heap_only` releases heap memory without trimming the working set; `full` is the default since 1.6.1. Commit size falls by whatever the heaps could decommit; it is reported, not judged.

Step 4 — GUI checklist

The GUI writes to `gui_output\`. Tick the outputs you want (CSV at least) and press **Start Auto**.

- ☐ Window opens; *View* menu switches light/dark theme (unticked check boxes stay visible in dark).
- ☐ After the first poll: two tabs **Meter 127.0.0.2** and **Meter 127.0.0.3**; at the top of each: `reports 127.0.0.x (matches)`, serial `EMU10000` / `EMU10001`.
- ☐ Plots per unit (Voltage, Current, Real power, …) update every poll (the GUI's sample period is at least 5 s); the newest point stays at the right edge (*Follow newest* ticked).
- ☐ Move the mouse over a plot: a dashed crosshair appears in **every** plot of that meter; the *At cursor* column fills in; values of 127.0.0.3 are about 15 % higher than those of 127.0.0.2.
- ☐ Mouse wheel over a plot **scrolls the tab**; **Ctrl + wheel zooms** the plot and *Follow newest* unticks; the next polls do not move the zoomed view; ticking *Follow newest* again jumps back to the newest data.

- ☐ Untick a trace in a legend column → that line disappears; tick it → it returns.
- ☐ *View: by table* and back to *by unit* works; *Show: last 10 min* narrows the time span.
- ☐ Right-click a plot → *Export…* → CSV writes that plot's data.
- ☐ Status line shows `Store … | RSS … MB (commit … MB) | system RAM … %`.
- ☐ **Flush RAM Now** → log line `OS release [heap_optimize ok, working_set_trim ok]; RSS a -> b MB, commit c -> d MB`, and the python.exe working set in Task Manager drops.
- ☐ Open `gui_output\csv\ABMeter_127_0_0_2_Real_Time_Power_Table.csv` **in Excel** while polling → exactly **one** warning "is open in another program", no errors, polling continues.
- ☐ With the CSV still open in Excel, press **Save Now** → a `…_rescue.csv` file appears next to it.
- ☐ Close Excel → within one poll the rescue file disappears and the CSV continues without gaps or duplicates (rows in time order).
- ☐ **Open in Veusz** (needs `env_veusz` or an installed Veusz) opens `ABMeter_127_0_0_2.vszh5`; it keeps growing while polling (reload in Veusz).
- ☐ Close the window while polling → it waits for the final writes, then closes.

Step 5 — headless stop

Press **Ctrl+Break** (or Ctrl+C) in the monitor window: the log must end with `Final writes done` and the exit code must be 0. Answer **N** if cmd asks "Terminate batch job (Y/N)?". Many laptops have no Break key — Ctrl+C does the same here; Ctrl+Break matters for supervisors that send `CTRL_BREAK_EVENT`.

What to send back

1. `results\AB_TestResults_<computer>_<time>.zip` (from step 2 — the main one).
2. `results\task_manager_check.json` (step 3) and, if you like, a Task Manager screenshot.
3. Which GUI checklist items passed or failed, and anything odd.

Troubleshooting

Symptom	Fix
"Windows protected your PC" / security warning per `.bat`	unblock the 7z (see *Before you start*) and extract again
Setup: "path contains a space"	move the kit to e.g. `C:\ABTest`
Setup: neither mamba/conda nor Python 3.12 found	install Miniforge (https://github.com/conda-forge/miniforge), then run step 1 again
`system.json` shows `loopback_127_0_0_2: false`	the emulator cannot use 127.0.0.2; report it — the tests need it
A firewall prompt	should not appear (everything binds to 127.0.0.x); allowing "private networks" is harmless
Tests slow	antivirus scanning the many small writes; normal, the time limits allow for it
Port 18080 in use (steps 4–5)	edit the two `.bat` files: change both `18080` to another free port
Rebuild the environment	delete `env\` (and `env_veusz\`), run step 1 again

Verified

- **txrx03** (Windows 11 26H2, i9, 32 GB), 1.6.2 kit, 2026-10-26: 7 of 8 suites passed. Memory flush: working set 298 → 4 MiB. Headless soak: 591 polls = 591 rows in CSV, Markdown, .vszh5, XLSX and FITS, Ctrl+Break stop clean. The GUI suite failed with the MKL crash above; with OpenBLAS the GUI ran 80 polls writing every output.

Before the first shipment (2026-10-26)

- Linux, monitor 1.7.0: all eleven suites pass (`run_all_tests.py --quick`) on Python 3.12.3 (PySide6 6.12.0, pyqtgraph 0.14.0) and 3.7.12 (PySide6 6.2.4, pyqtgraph 0.12.4); `test_live_plots.py` also with PySide6 6.11.2, the version this kit installs. The kit environment with pyqtgraph 0.14.0 solves for win-64 (dry run), still with OpenBLAS.
- Linux, Python 3.12.3: all nine suites of 1.6.4 pass (`run_all_tests.py --quick`); the suites of 1.6.5 pass one by one on Python 3.12.3 and 3.7.12.
- Windows build of Python 3.12.15 (same package versions as `environment-windows-test.txt`) under Wine 9.0: setup script, memory release, file locks (real `CreateFileW` locks), memory check and the GUI soak pass. Not checkable under Wine, so checked for the first time on your PC: the real Veusz render, the Ctrl+Break stop, and the Task Manager drop itself.

Files

File	Purpose
`1_` … `5_*.bat`	the steps above
`_env.bat`	shared settings for the steps (finds `env\python.exe`; PATH unchanged)
`environment-windows-test.txt`	exact conda-forge versions for `env\`
`environment-veusz.txt`	Veusz 4.2.1 for `env_veusz\`
`app\` (or the folder above `windows\`)	the monitor 1.7.0 with `ab_live_plots.py`, exporter 0.2.0, dashboard builder, meter check, writer, tests, docs, `monitoring\`
`app\tests\run_all_tests.py`	the runner behind step 2 (options: `--quick`, `--only`, `--skip`, `--soak-minutes`, `--gui-seconds`, `--veusz-exe`)